

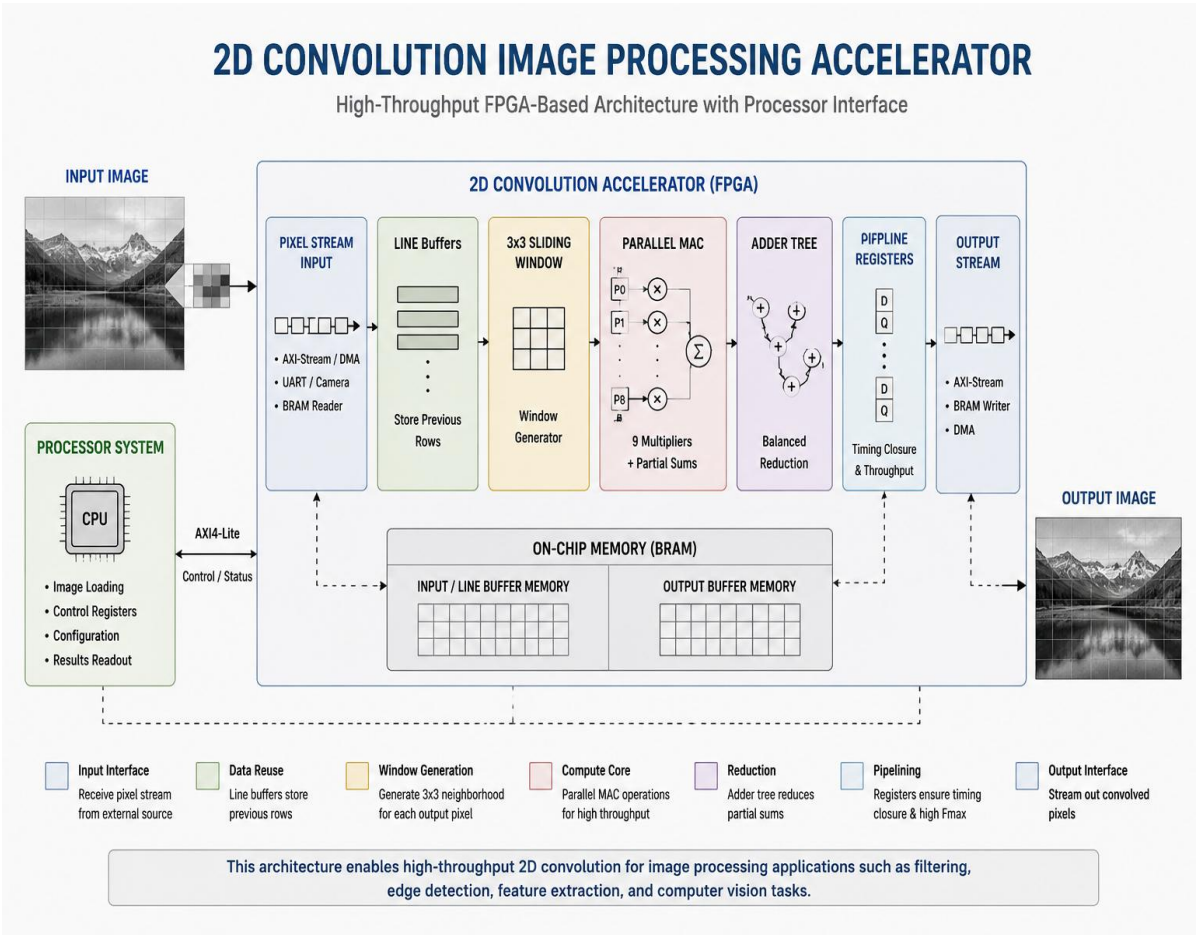
Technical Design Proposal

2D Convolution Image Processing Accelerator on FPGA

Initial Technical Design Specification — Version 0.1

Digital Design & FPGA Project Team
Three-Week Development Scope
August 2026

Architecture status: **Candidate architecture under investigation — not yet frozen.**



Executive Summary

This document presents the initial technical design proposal for a 2D Convolution Image Processing Accelerator implemented on FPGA. Two-dimensional convolution is a fundamental operation in image processing and is also a core computational pattern in convolutional neural-network workloads. The project is intended to demonstrate the transformation of a mathematical/software algorithm into a verified, resource-aware RTL hardware architecture.

The development flow is intentionally divided into four layers: a MATLAB software reference model, an RTL hardware implementation, FPGA synthesis/implementation, and output comparison between software and hardware. MATLAB will act as an independent golden reference rather than attempting to reproduce the internal FPGA microarchitecture.

The initial baseline is a 3×3 kernel operating on 8-bit grayscale image data, with stride 1 and valid padding. Multiple hardware architectures will be studied before the final architecture is frozen: direct convolution, sliding-window/line-buffer designs, streaming designs, fully parallel MAC, resource-shared MAC, and a more advanced systolic approach.

The current leading candidate is a streaming/sliding-window architecture using line buffers, parallel multiply-accumulate operations, a balanced adder tree, and pipeline registers. This is explicitly a candidate rather than a final commitment. The final selection will depend on the available FPGA device, synthesis results, timing/resource measurements, project duration, and supervisor feedback.

A Zynq PS/PL hardware/software co-design path and PC-based data transfer are considered optional extensions. The core project must remain complete using a preloaded memory/test-vector method so that communication integration cannot block the three-week RTL/FPGA deliverable.

1. Introduction

2D convolution is widely used for edge detection, smoothing, sharpening, noise reduction, feature extraction, and as a fundamental operation in CNNs. Although the mathematical operation is compact, an FPGA implementation exposes important hardware questions: how much computation should be parallelized, how image data should be stored and reused, how arithmetic precision should be represented, and where pipeline registers should be inserted.

The engineering value of this project is therefore not only the final convolution result. The project will compare hardware mappings and quantify the trade-offs between throughput, latency, memory bandwidth, LUT/FF/BRAM/DSP usage, timing, and implementation complexity.

The central engineering question is:

How can 2D convolution be mapped onto FPGA hardware to achieve a useful balance between throughput, latency, memory traffic, resource utilization, and implementation risk within three weeks?

2. Problem Statement

For a $K \times K$ convolution, every output pixel requires K^2 pixel values, K^2 coefficients, K^2 multiplications, and an accumulation. In the initial 3×3 case, nine pixel/coefficient products must be reduced to one output value.

A naive implementation treats each output window independently. However, neighboring 3×3 windows overlap heavily. Reading every window directly from image memory can therefore cause unnecessary memory traffic and make data movement a limiting factor even when the arithmetic datapath is fast.

1. Computation problem: determine the useful degree of parallelism and whether multiplication resources should be fully parallel, partially parallel, or time-multiplexed.
2. Data-movement problem: generate overlapping windows efficiently while reusing pixels already available on-chip.
3. Timing problem: keep multiplier and adder-tree paths short enough to meet the target clock after synthesis.
4. Numerical problem: define signedness, product width, accumulator width, overflow, and scaling consistently between MATLAB and RTL.

3. Initial Project Specifications

The following values define the initial baseline for the study. They are assumptions for scope control, not final architectural commitments.

Parameter	Initial Specification
Operation	2D convolution / image filtering
Initial kernel	3×3
Image	Grayscale, single channel
Pixel format	8-bit unsigned
Kernel coefficients	Signed integer; fixed-point to be evaluated
Stride	1
Padding	Valid / no zero-padding initially
HDL	Verilog
Software reference	MATLAB
Initial verification	MATLAB ↔ RTL simulation ↔ FPGA
Baseline data input	Preloaded .mem / test vectors
Optional host	Python or MATLAB
Optional SoC	Zynq PS/PL
Primary metrics	Fmax, latency, throughput, LUT, FF, BRAM, DSP
Target FPGA	TBD — determined by available development board

4. Mathematical Background

For the initial hardware specification, the operation follows the common image-processing correlation convention: the kernel is applied in its given orientation rather than being spatially flipped. This convention must be kept identical in MATLAB and RTL.

$$Y(i,j) = \sum_m \sum_n I(i+m,j+n) \times W(m,n), \quad m,n = 0 \dots K-1$$

For K=3, one output pixel is the sum of nine products. With valid padding and stride 1, the output dimensions are:

$$\text{Output} = \text{floor}((\text{Input} + 2p - K) / s) + 1$$

3×3 Image Window	×	3×3 Kernel	→	
10 20 30 40 50 60 70 80 90		1 0 -1 1 0 -1 1 0 -1		$Y = \sum P_i K_i$

Each output pixel is a sum of nine pixel-coefficient products.

Figure 1 — 3×3 convolution as a sum of nine pixel-coefficient products.

5. MATLAB Software Reference / Golden Model

5.1 Purpose

MATLAB is used as an independent mathematical reference. It should express what the algorithm must produce, not how the FPGA internally produces it. This separation makes it possible to compare the hardware against an independently constructed expected result.

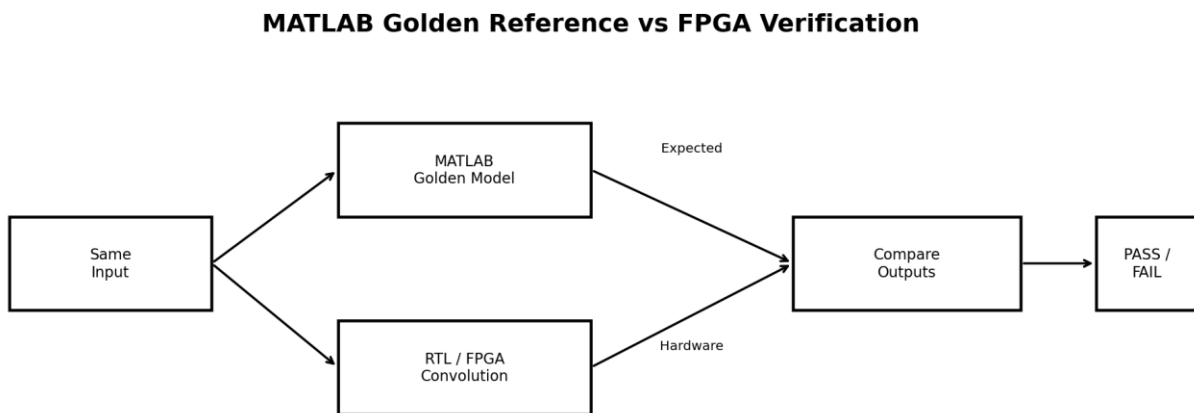


Figure 2 — Independent MATLAB reference and FPGA implementation compared using the same input.

5.2 Reference model flow

- Read a test image.
- Convert to grayscale when required.
- Convert to the numerical representation used by the reference.
- Define one or more kernels.
- Compute the expected output using MATLAB.

- Generate expected test vectors and, when useful, intermediate values.
- Export data for RTL simulation.
- Compare RTL/FPGA results against the golden output.

5.3 MATLAB versus Simulink

A plain MATLAB script or function is sufficient for the core reference model. Simulink is not required because the purpose of the model is to provide a trusted mathematical result and test-vector generator, not to reproduce the FPGA block diagram graphically.

5.4 Optional Python role

Python can optionally handle image file loading, grayscale conversion, pixel formatting, generation of .mem or binary files, and host-to-FPGA communication. It is not necessary for the core project and should not become a separate learning project.

6. Image Data Representation and Input Methods

An 8-bit grayscale image is already numerical data: each pixel is represented by an integer from 0 to 255. The host-side preprocessing step converts the image into an ordered stream or memory file. The FPGA does not need to understand JPEG/PNG formats in the core design.

6.1 Baseline — preloaded memory

Image → MATLAB/Python → pixel values → .mem → simulation/BRAM. This is the recommended first input method because it removes communication complexity from the critical path of the project.

6.2 Optional — PC to FPGA

Image → Python/MATLAB → byte stream → UART or another interface → FPGA. This allows changing images without rebuilding the convolution datapath, but transfer time and interface debugging must be considered.

6.3 Optional — Zynq PS/PL

PC/software → Zynq PS → DDR → AXI/DMA → FPGA PL convolution accelerator. This provides a genuine hardware/software co-design demonstration but adds AXI, DMA, memory-buffer, cache, and software-control complexity.

Optional Zynq HW/SW Co-Design Extension

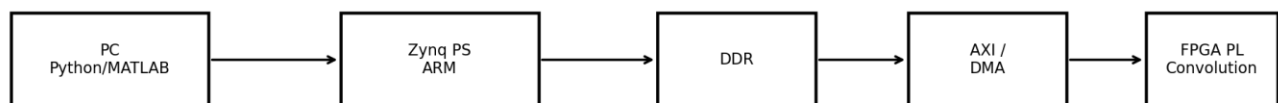


Figure 3 — Optional Zynq PS/PL system-level extension. This is not required for the core three-week deliverable.

7. System-Level Architecture

The candidate accelerator is organized as a streaming datapath. The input source may later be a memory reader, UART receiver, or DMA interface; the convolution core itself should remain independent of that choice.

Candidate Streaming FPGA Convolution Architecture

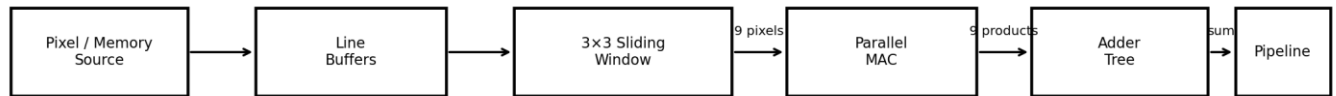


Figure 4 — Candidate streaming architecture: source, line buffers, window generation, MAC, reduction and pipeline.

The key architectural principle is to separate data movement from computation. The line-buffer/window subsystem creates the 3×3 neighborhood, while the arithmetic subsystem consumes that neighborhood and produces the output. This modularity allows alternative memory or communication interfaces without rewriting the convolution arithmetic.

8. Detailed Sub-Block Explanation

8.1 Input / Memory Interface

The input interface must preserve pixel ordering and provide a valid pixel stream to the window generator. The first implementation can use a simple memory reader. A more advanced implementation may use a streaming interface or DMA.

8.2 Line Buffers

For a 3×3 window, two previous rows must be available while the current row is being received. Line buffers provide this vertical reuse. They can be implemented with registers, distributed memory, BRAM, or FIFO-like structures depending on image dimensions and FPGA resources.

8.3 Sliding-Window Generator

Horizontal shift registers combine the current pixel with values retained from the previous columns, producing a new 3×3 neighborhood as the stream advances. After the initial fill period, neighboring windows share most of their pixels instead of re-reading them from memory.

8.4 Kernel Storage

For the baseline project, the nine coefficients can be stored in registers and loaded before processing a frame. A configurable-kernel interface can be added later.

8.5 Multiply-Accumulate

The datapath multiplies the nine window pixels by the nine kernel coefficients. This can be fully parallel, partially parallel, or resource-shared.

8.6 Adder Tree

The nine products should preferably be reduced through a balanced tree rather than a long serial chain. A balanced tree reduces combinational depth and provides a natural place for pipeline registers.

8.7 Pipeline and Valid Control

Pipeline registers can separate multiplication and reduction stages. A valid signal must travel with the corresponding data so that output validity remains aligned with the pipeline latency.

9. Architecture Alternatives

9.1 Architecture A — Direct / Naive Convolution

Architecture A — Direct / Naive Convolution

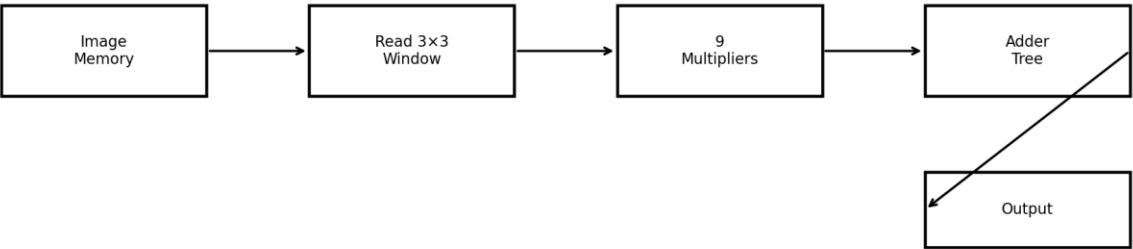


Figure 5 — Direct architecture: each output window is read and processed independently.

Concept: read a 3×3 window, perform nine multiplications, reduce the products, and produce one output.

Advantages	Disadvantages
Very simple baseline	Repeated memory accesses
Easy to understand and verify	Poor data reuse
Useful as a reference architecture	Memory bandwidth can dominate

Role: baseline for comparison, not necessarily the final implementation.

9.2 Architecture B — Sliding Window + Line Buffers

Architecture B/C — Sliding Window + Streaming

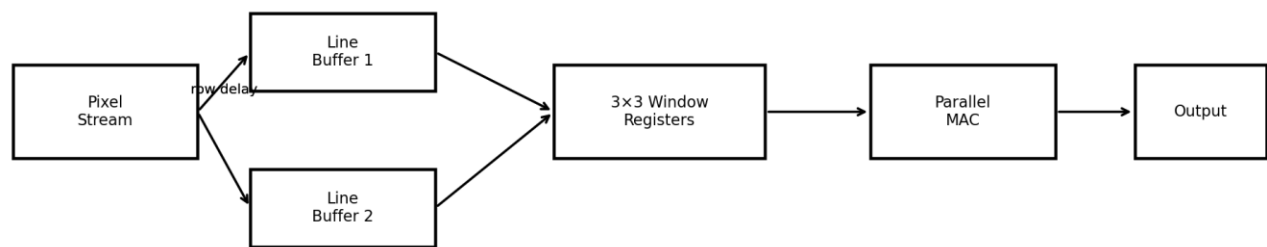


Figure 6 — Sliding-window and line-buffer architecture.

Advantages	Disadvantages
High pixel reuse	More control/boundary logic
Lower memory traffic	Harder RTL verification
Natural FPGA mapping	Requires careful row/column handling
Suitable for streaming	Line-buffer resource must be sized correctly

9.3 Architecture C — Streaming Convolution

A streaming design treats pixels as a continuous sequence. Once the window has been filled, the design can potentially produce an output every cycle, subject to the selected arithmetic parallelism and pipeline.

Advantages	Disadvantages
High throughput potential	Boundary and valid timing are more complex
Good data reuse	Debugging requires cycle-level reasoning
Interface-friendly	Upstream/downstream flow control may be needed

9.4 Architecture D — Fully Parallel MAC

Parallel MAC and Balanced Adder Tree

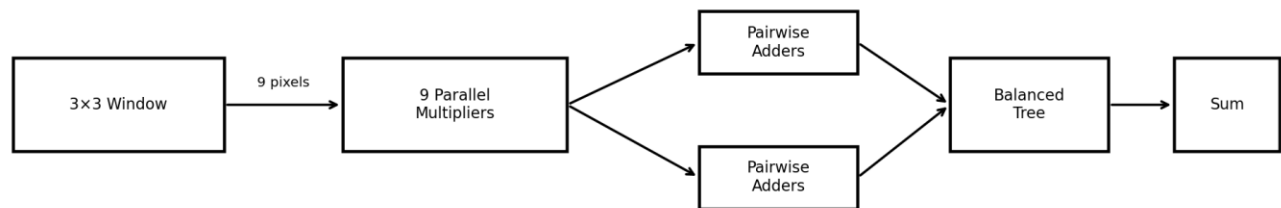


Figure 7 — Nine parallel multipliers followed by a balanced adder tree.

Advantages	Disadvantages
High computational throughput	High DSP/LUT usage
Simple datapath concept	More routing and switching
Good use of FPGA DSP blocks	May not scale efficiently to large kernels

9.5 Architecture E — Resource-Shared / Time-Multiplexed MAC

A smaller number of multipliers is reused across multiple cycles. This trades throughput and latency for reduced hardware resources.

Advantages	Disadvantages
Lower area	More cycles per output
Fewer DSPs required	Lower throughput
Useful for resource-constrained devices	More control complexity

9.6 Architecture F — Systolic / Processing-Element Array

A systolic design distributes computation across processing elements and exploits regular data movement between neighboring elements. It is attractive for larger accelerator systems but introduces additional control and mapping complexity.

Advantages	Disadvantages
High parallelism	High implementation complexity
Strong data reuse	More difficult verification
Scalable concept	Potentially excessive for a three-week 3×3 core

10. Memory Architecture Alternatives

Memory Structure	Best Use	Advantages	Disadvantages
BRAM	Image storage / line buffers	Dense on-chip storage, predictable	Limited quantity and port constraints
Registers	Small windows / pipeline	Very fast, simple	Poor scaling for large images
Shift registers	Horizontal window movement	Excellent for streaming windows	Not suitable for full-frame storage
Distributed RAM	Small buffers / coefficients	Flexible LUT-based memory	Consumes LUT resources
Line buffers	Previous image rows	High data reuse	Requires careful addressing/control

The final memory architecture should be selected together with the dataflow. A fast MAC array cannot deliver its potential throughput if the memory subsystem cannot provide the required pixels each cycle.

11. Arithmetic Design Alternatives

11.1 Integer

Integer arithmetic is the simplest baseline and is attractive for a short student project. It is easy to reason about and verify, but fractional filter coefficients require scaling or quantization.

11.2 Fixed-Point

Fixed-point supports fractional coefficients while avoiding floating-point hardware. The design must define integer and fractional bits, rounding, saturation, and acceptable numerical error.

11.3 Floating-Point

Floating-point is technically possible but is not recommended for the initial three-week core because of higher hardware cost and verification complexity. It can be considered only if the target application requires it.

11.4 Width and overflow

If pixels are 8-bit unsigned and coefficients are signed, products become wider than the input pixel. The accumulator must be sized for the worst-case sum. MATLAB and RTL must use explicitly matched widths and signedness so that overflow behavior is deterministic.

12. Multiplication Implementation

Method	Advantages	Disadvantages	When to Consider
DSP-based	Fast, dedicated hardware, natural for MAC	Consumes DSP blocks	Preferred when DSP budget permits
LUT-based	Flexible, no DSP consumption	Can use many LUTs and affect timing	Useful when DSPs are scarce or coefficients are simple
Shift/add for special coefficients	Very low cost for powers-of-two/sparse coefficients	Only works for restricted kernels	Useful for selected fixed filters

The final choice should be based on synthesis rather than assumption.

13. Adder Tree and Pipeline

Nine products can be added serially, but a balanced reduction tree reduces logic depth. Pipeline registers may be inserted between multiplication, partial-sum, and final-sum stages.

Conceptual Pipeline

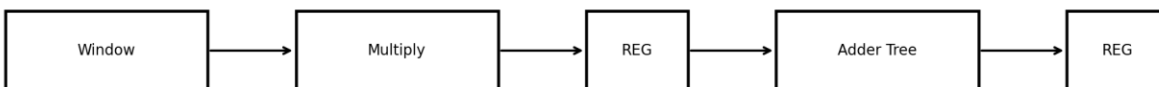


Figure 8 — Conceptual pipelined datapath. Exact stage boundaries depend on synthesis and timing results.

Latency is the number of cycles from an accepted input/window to its corresponding output. Throughput is the rate at which outputs can be produced once the pipeline is full. A multi-cycle latency design can still achieve one output per cycle.

14. Data Flow and Cycle-Level Behavior

Cycle	Window / Input	Processing	Output
C0	Pixel stream begins	Line-buffer fill	—
C1	First valid 3×3 window	Multiply stage	—
C2	Next window	Adder / reduction	—
C3	Next window	Pipeline	Output A
C4	Next window	Pipeline	Output B
C5+	Continuous windows	Steady state	One output per selected throughput interval

This timing is conceptual. The actual latency and output interval will be measured after the final RTL pipeline is selected.

15. Verification Strategy

Verification is designed as an independent chain rather than a single testbench.

5. Verify small convolution examples manually.
6. Generate golden outputs in MATLAB.
7. Run the same inputs through RTL simulation.
8. Compare RTL output against MATLAB output.
9. Program the FPGA and run the same test vectors.
10. Capture hardware output and compare it against MATLAB.
11. Record mismatches, latency, and performance.

15.1 Test classes

- Small deterministic images for hand calculation.
- Identity kernel.
- Horizontal/vertical edge-detection kernels.
- Box blur / smoothing kernels.
- Sharpening kernels.
- Random images and randomized coefficients within the defined numeric range.
- Boundary and minimum-size images.
- Cases that exercise signed negative coefficients.
- Overflow/maximum-value cases when fixed-width arithmetic is used.

16. Performance and Resource Metrics

Category	Metric
Functional	Output correctness / mismatch count
Timing	Fmax / critical path
Latency	Input-to-output cycles
Throughput	Pixels per cycle and pixels per second
Logic	LUT count / utilization
Registers	FF count / utilization
Memory	BRAM count / utilization
Arithmetic	DSP count / utilization
System (optional)	Transfer time / end-to-end time

No final performance number is claimed in this initial proposal. The final numbers must come from synthesis, implementation, and hardware measurements on the actual target device.

17. Architecture Comparison and Decision Matrix

The following is an initial qualitative comparison. It is a decision aid, not a substitute for synthesis.

Architecture	Area	Throughput	Memory Efficiency	Control	Timing Potential	3-Week Risk
Direct	Low–Medium	Low–Medium	Poor	Low	Medium	Low
Sliding Window	Medium	High	High	Medium	High	Medium
Streaming	Medium	High	High	Medium	High	Medium
Parallel MAC	High	Very High	Depends	Medium	High	Medium
Resource Shared	Low	Low–Medium	Depends	Medium–High	Medium	Medium
Systolic	Medium–High	Very High	High	High	High	High

During Week 1, the team should weight these criteria according to supervisor priorities. For example, throughput, timing, and three-week feasibility may receive higher weights than absolute minimum area.

18. Candidate Architecture Selection

The current leading candidate is a 3×3 grayscale streaming/sliding-window architecture with line buffers, parallel MAC, balanced adder tree, and pipeline registers.

Candidate Streaming FPGA Convolution Architecture

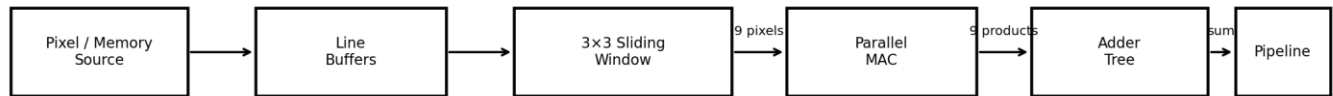


Figure 9 — Current candidate architecture. Final selection remains open until Week 1 study and supervisor discussion.

Reasons for selecting it as the leading candidate for investigation:

- Strong reuse of overlapping image pixels.
- Meaningful FPGA-specific optimization rather than a direct software translation.
- High throughput potential.
- Clear mapping to BRAM/register/DSP resources.
- Good opportunity to study timing and pipelining.
- Still realistic for a three-week core implementation.

This is not a final commitment. If synthesis or the target board indicates that a different architecture is more appropriate, the project should switch before the RTL is deeply committed.

19. Three-Week Execution Plan

Week 1 — Study, Reference Model, and Architecture Selection

- Understand 2D convolution mathematically.
- Build MATLAB golden reference.
- Define data widths, signedness, stride and padding.
- Study direct, sliding-window, streaming, parallel, resource-shared and systolic architectures.
- Study BRAM, registers, line buffers and DSP/LUT multiplication.
- Choose the architecture based on constraints and supervisor feedback.
- Freeze detailed block diagram and interface specification.

Week 2 — RTL and Simulation

- Implement input/memory interface.

- Implement line buffer and sliding-window logic if selected.
- Implement kernel storage/configuration.
- Implement multiplier/MAC datapath.
- Implement balanced adder tree and pipeline.
- Implement valid/control logic.
- Build RTL testbench.
- Compare against MATLAB golden results.

Week 3 — FPGA and Measurement

- Synthesize and implement.
- Analyze timing and resource utilization.
- Program FPGA and execute test vectors.
- Capture output and compare against MATLAB.
- Measure Fmax, latency, throughput, LUT/FF/BRAM/DSP.
- Document results and optimization opportunities.
- If the core is stable early, attempt optional PC or Zynq integration.

20. Team Division

Team Member	Primary Responsibility
Member 1	Algorithm, MATLAB reference, test vectors, verification
Member 2	RTL datapath: window, line buffers, MAC, adder tree, pipeline
Member 3	FPGA integration, synthesis, timing/resource analysis, hardware testing

The division is for parallel progress, not isolation. Every member should understand the full system and be able to explain the architecture and verification flow.

21. Risks and Mitigation

Risk	Impact	Mitigation
Over-scope	Project may not finish	Freeze 3×3 grayscale core first
Memory bottleneck	Low throughput	Use line buffers/sliding window and measure bandwidth
Timing failure	Low Fmax	Balanced adder tree and pipeline
Numeric mismatch	Incorrect comparison	Define exact widths/signedness and independent MATLAB model
Communication delay	Integration consumes schedule	Keep .mem method as mandatory baseline
Zynq complexity	AXI/DMA debugging consumes time	Treat Zynq as optional extension
Insufficient FPGA resources	Architecture cannot fit	Compare parallel vs resource-shared options early

22. Tools and Requirements

- Verilog RTL toolchain.
- FPGA vendor tool such as Vivado or the appropriate equivalent for the selected board.
- MATLAB for golden reference and analysis.
- RTL simulator such as Questa/ModelSim/Vivado Simulator as available.
- Optional Python/NumPy for host-side data formatting and communication.
- Optional Vitis/embedded software flow if a Zynq extension is implemented.

- Target FPGA development board — **to be confirmed with the supervisor.**

23. Supervisor Discussion Points

- 12. Which FPGA development board/device should be the target?**
- 13. Should the initial image be grayscale, or is RGB required?
- 14. What image resolution is expected?
- 15. Should the kernel remain fixed at 3×3 or be configurable?
- 16. Should valid or same padding be required?
- 17. Is maximum throughput or minimum resource utilization the main priority?**
- 18. Are DSP blocks expected to be used?
- 19. Is fixed-point arithmetic required?
- 20. Is MATLAB acceptable as the golden reference?
- 21. Is live PC-to-FPGA image transfer required?
- 22. Is Zynq PS/PL HW/SW integration mandatory or optional?**
- 23. Which performance metrics should be reported?
- 24. Should architecture comparison be demonstrated experimentally or only discussed?
- 25. Is the three-week period intended for the complete working prototype?**

24. Future Extensions

- 5×5 and 7×7 kernels.
- RGB and multi-channel images.
- Multiple filters / CNN-style layers.
- Stride greater than one.
- Same padding.
- ReLU and pooling.
- AXI-Stream interface.
- DMA-based host/SoC transfer.
- Zynq PS/PL hardware/software co-design.
- Systolic or larger accelerator arrays.
- Quantized CNN inference.

25. Conclusion

This proposal defines a controlled three-week study and implementation path for a 2D convolution FPGA accelerator. The project begins with an independent MATLAB mathematical reference and progresses through architecture exploration, RTL design, simulation, FPGA implementation, and measured performance evaluation.

The central design decision is deliberately left open until the architecture alternatives are studied. Direct convolution provides a simple baseline; sliding-window and line-buffer techniques provide data reuse; streaming and parallel MAC structures provide throughput; resource sharing provides an area-oriented alternative; and systolic structures provide an advanced scalable direction.

The current leading candidate is the streaming/sliding-window architecture with line buffers, parallel MAC, balanced reduction, and pipelining. It will be treated as a hypothesis to validate rather than a predetermined answer.

The project therefore follows the engineering flow:

Algorithm → MATLAB Reference → Architecture Exploration → RTL → Verification → FPGA → Measurement

A Zynq-based hardware/software co-design is intentionally kept as an optional extension so that the core project remains achievable within the three-week schedule.

Appendix A — RTL Thinking Checklist

For every block, the team should answer the following before coding:

26. What operation or behavior are we implementing?
27. What hardware does that behavior imply?
28. Which Verilog/SystemVerilog construct best represents it?
29. Why is that construct appropriate?
30. What will synthesis likely infer: LUTs, FFs, BRAM, DSPs, or routing?
31. Where is the critical path?
32. How does the block affect latency and throughput?
33. How will MATLAB or the RTL testbench verify it?

Appendix B — Initial Architecture Questions to Study

- How many pixels can be supplied per cycle?
- How many outputs can be produced per cycle?
- How many DSPs are required for a fully parallel 3×3 design?
- Can multiplication be mapped efficiently to LUTs if DSPs are limited?
- How many line buffers are required for a 3×3 stream?
- What memory port behavior is required?
- Where should pipeline registers be inserted?
- What accumulator width guarantees no unintended overflow?
- How should negative coefficients be represented?
- How will border pixels be handled?
- How will valid/ready or valid-only timing be aligned?
- How will MATLAB and RTL use exactly the same numerical specification?